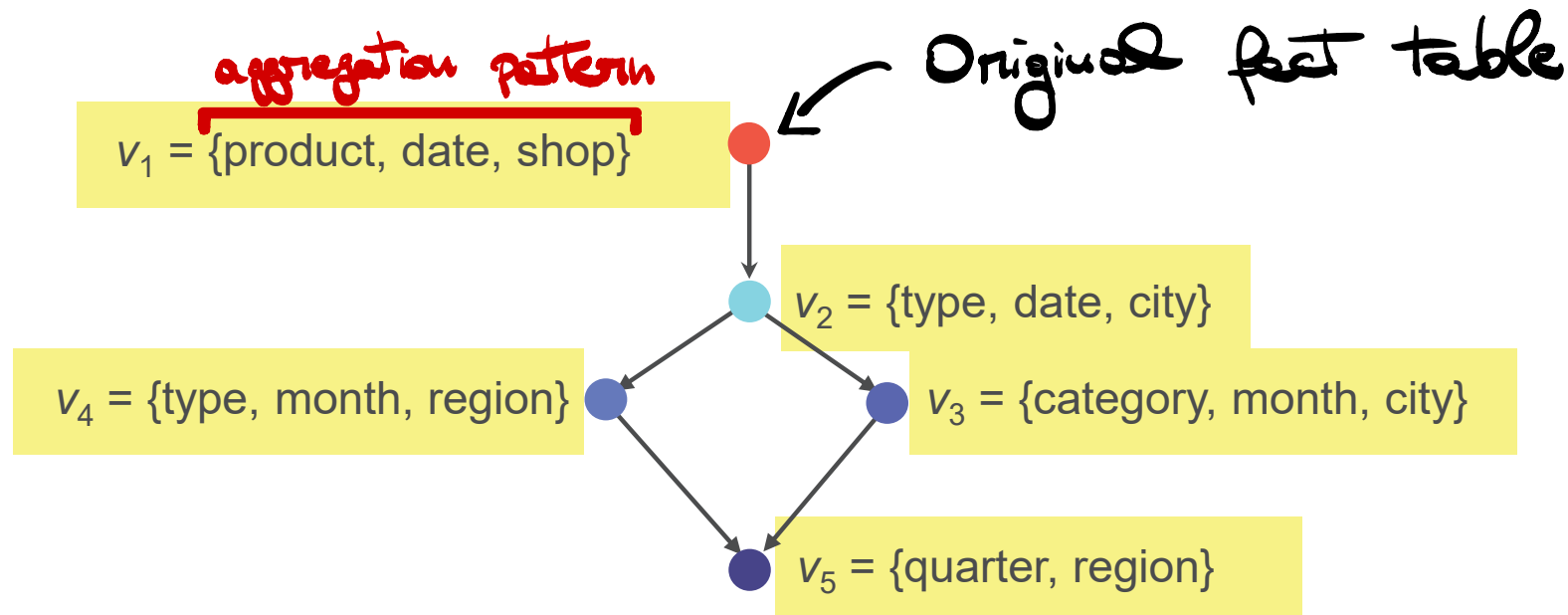


Materialized views

Elena Baralis
Politecnico di Torino

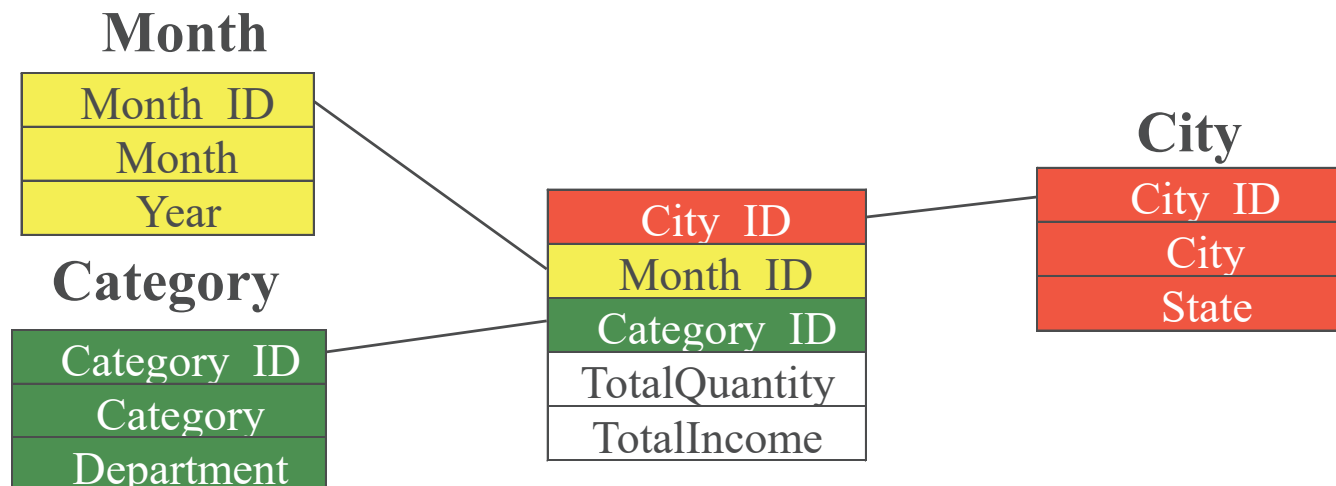
Materialized views

- Precomputed summaries for the fact table
 - explicitly stored in the data warehouse
 - provide a performance increase for aggregate queries



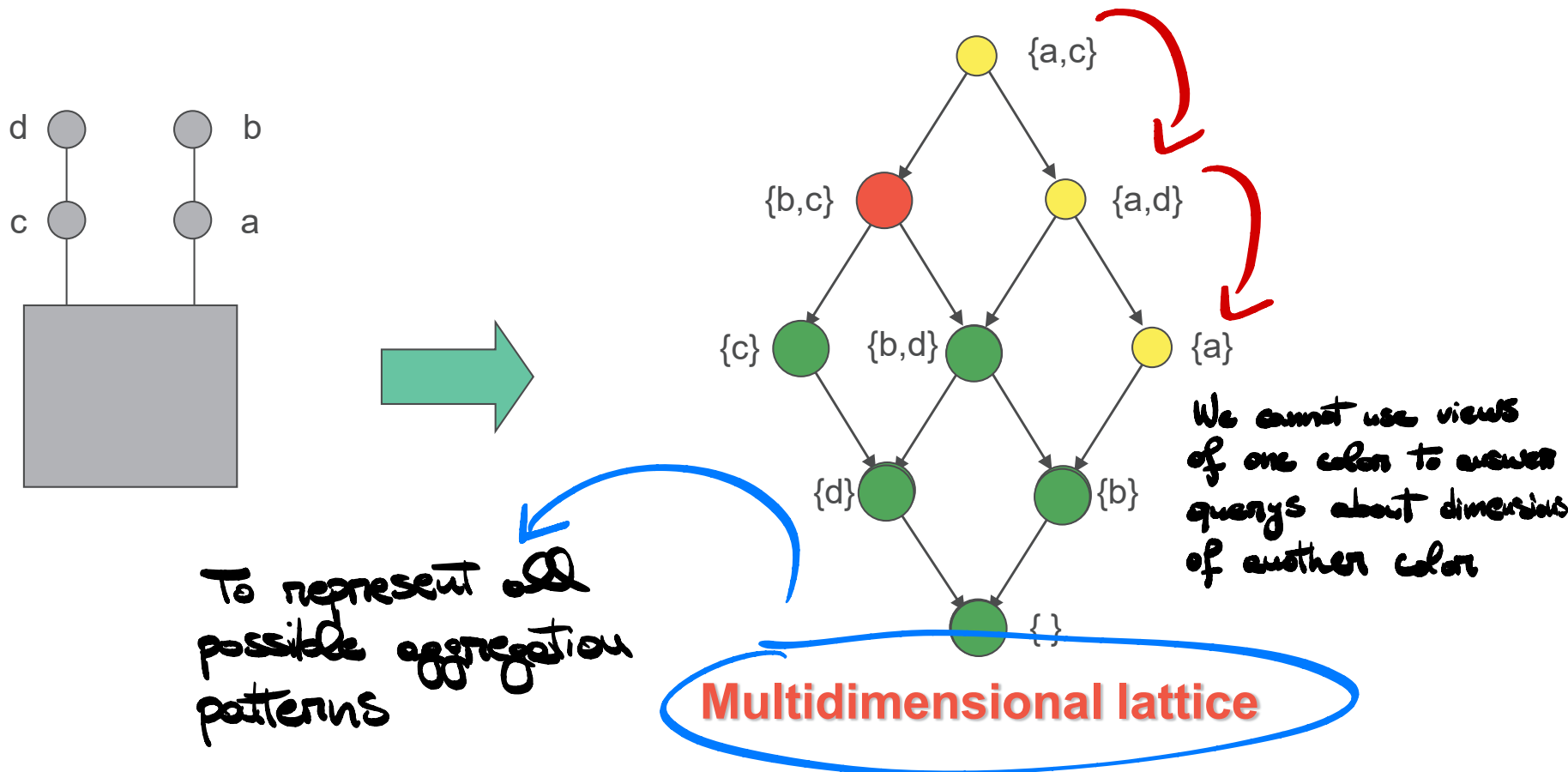
Materialized views

- Defined by SQL statements
- Example: definition of v_3
 - Starting from base tables or views with higher granularity
 - group by City, Category, Month
 - Aggregation (SUM) on Quantity, Income measures
 - Reduction of detail in dimensions



Materialized views

- Materialized views may be exploited for answering several different queries
 - not for all aggregation operators



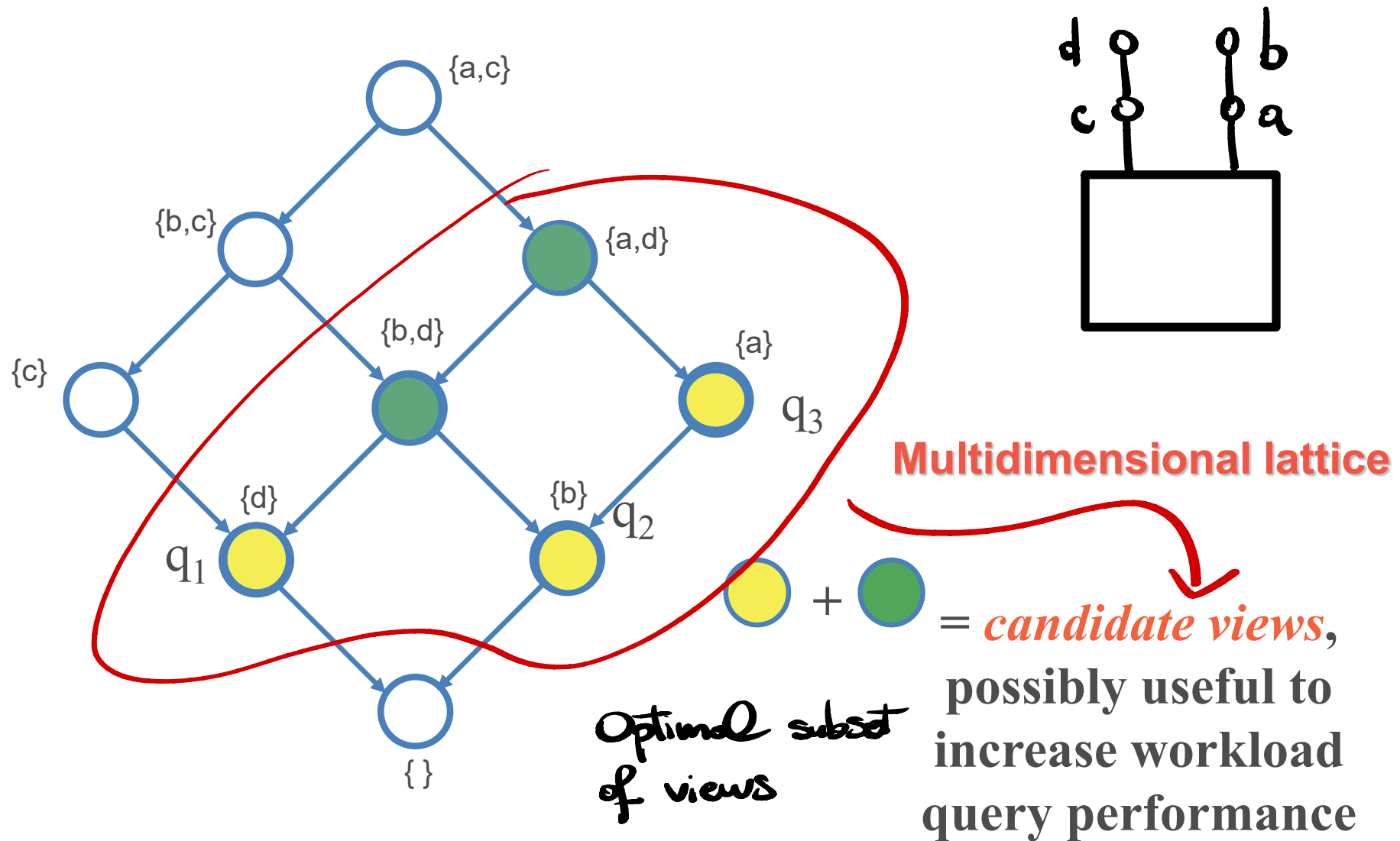
Materialized view selection

- Huge number of allowed aggregations
 - most attribute combinations are eligible
- Selection of the “best” materialized view set
- Cost function minimization
 - query execution cost
 - view maintenance (update) cost

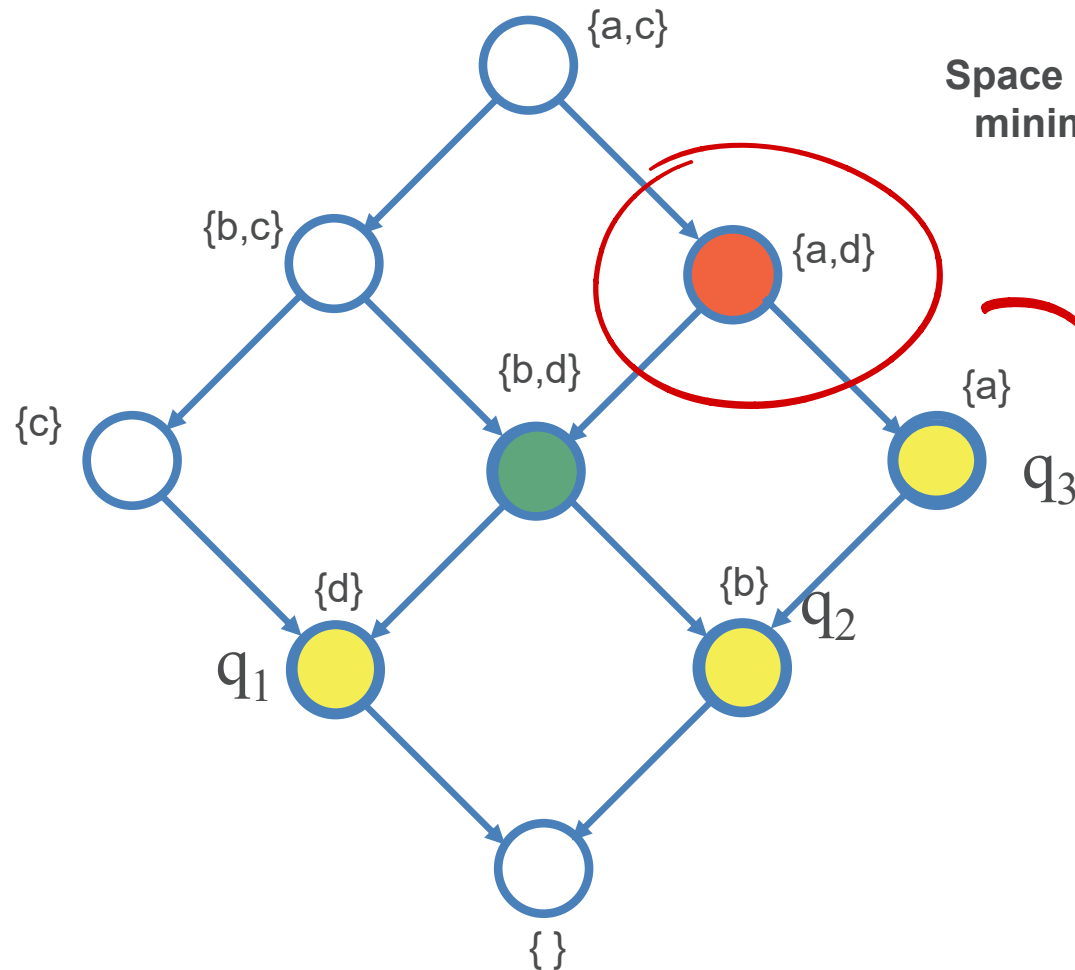
→ We are able to reduce the cost function
- Constraints
 - available space
 - time window for update
 - response time
 - data freshness

→ If the views are several, the time window may be not enough

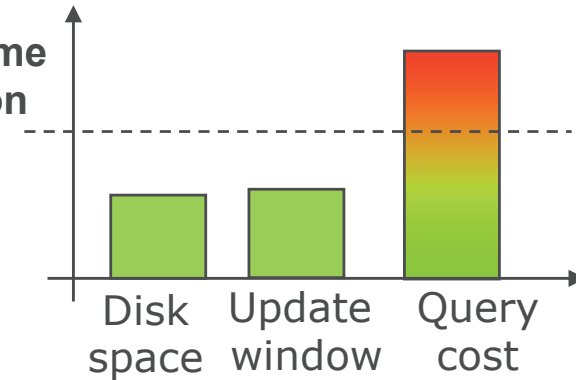
Materialized view selection



Materialized view selection

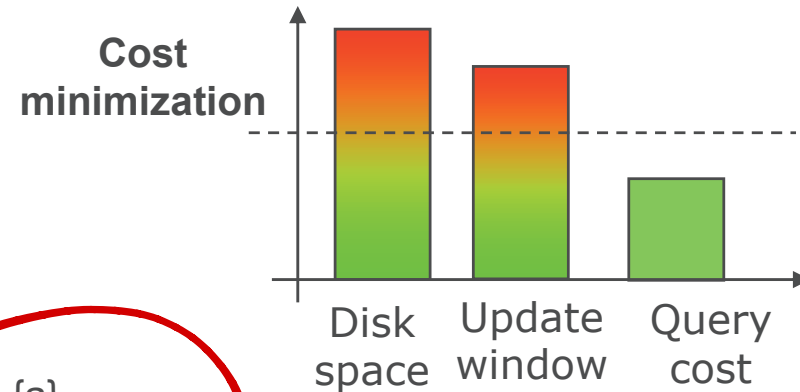
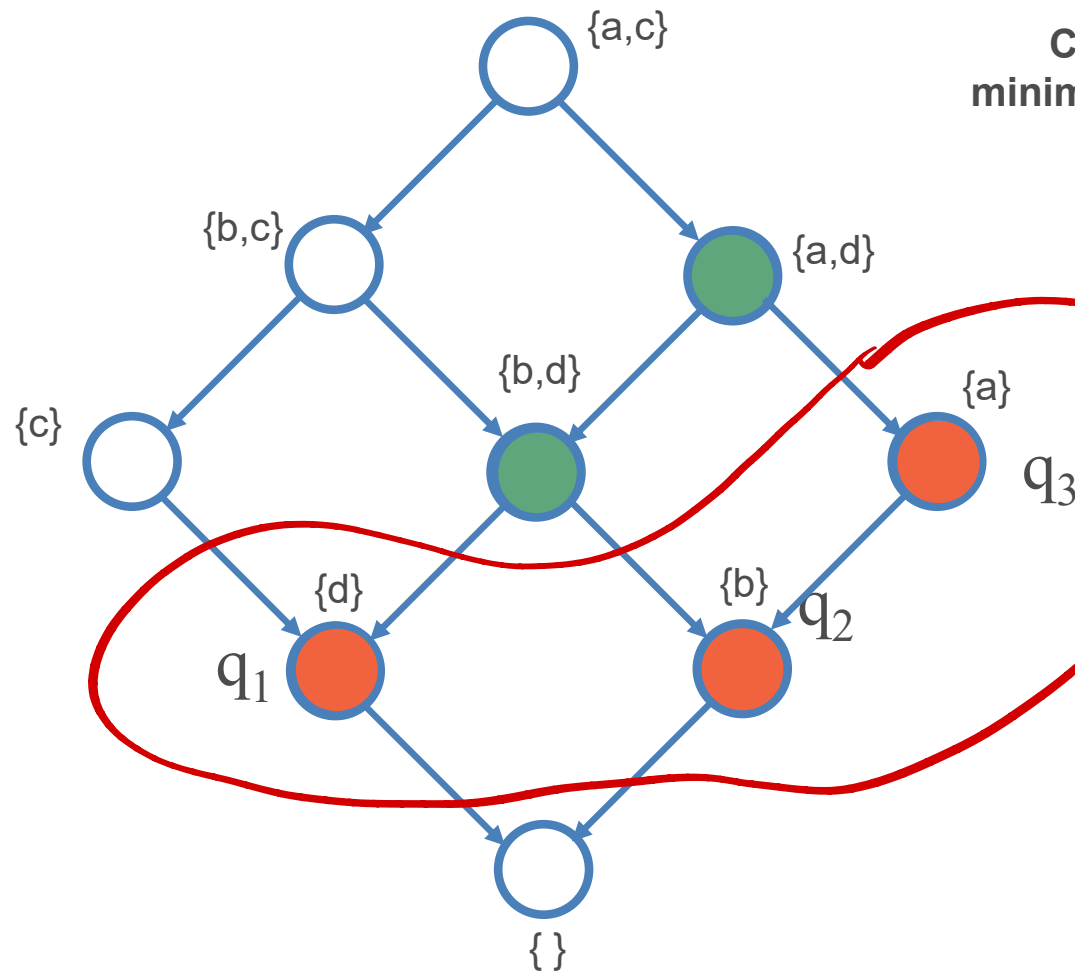


Space and time
minimization



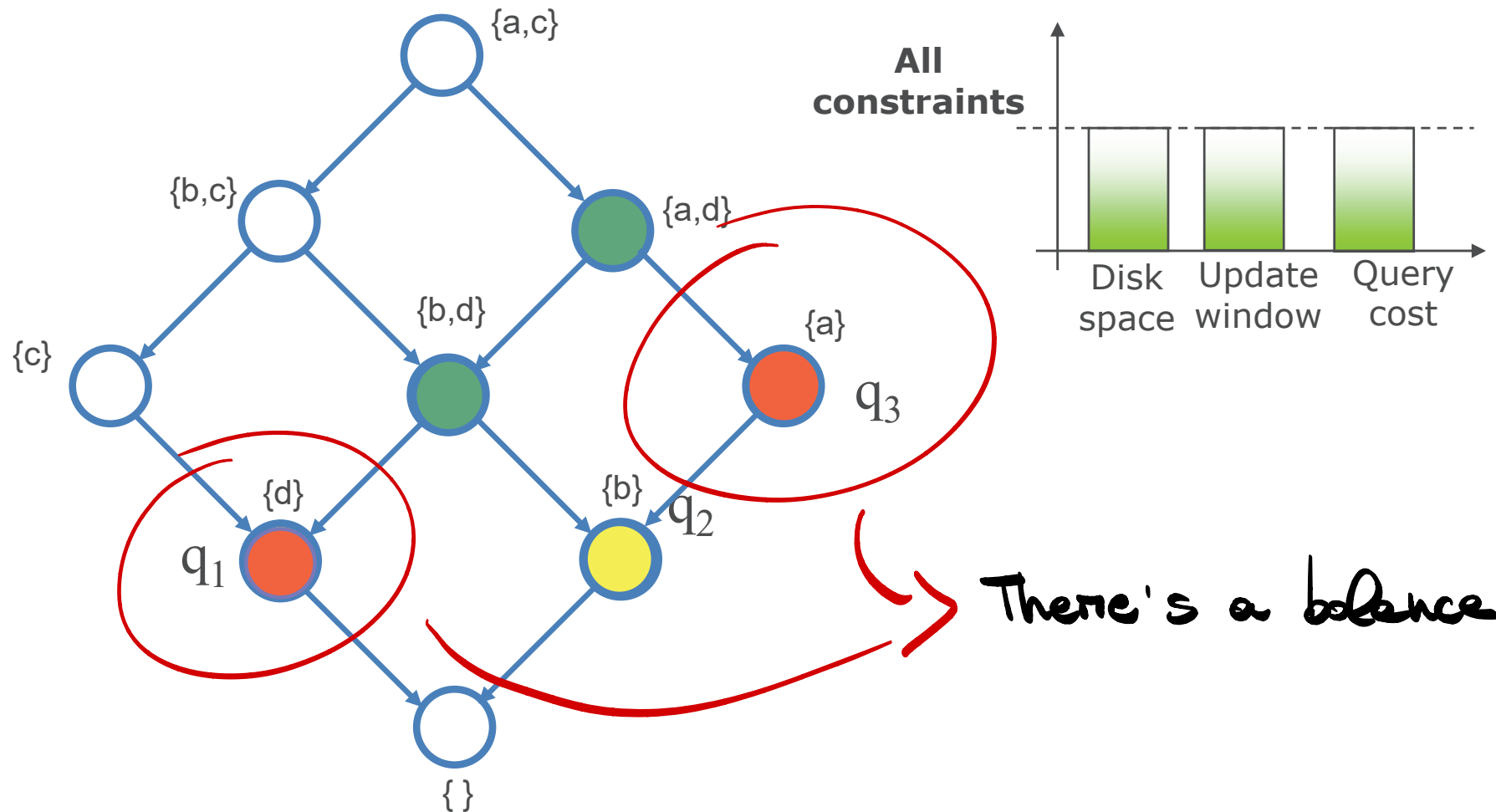
Using one single view, we can minimize disk space and update window

Materialized view selection



The query cost is minimized, but disk space and update window are not

Materialized view selection



Physical design

Elena Baralis
Politecnico di Torino

Physical design

- Workload characteristics
 - aggregate queries which require accessing a large fraction of each table
 - read-only access
 - periodic data refresh, possibly rebuilding physical access structures (indices, views)

- Physical structures

- index types different from OLTP
 - bitmap index, join index, bitmapped join index, ...
 - B⁺-tree index not appropriate for
 - attributes with low cardinality domains
 - queries with low selectivity
- materialized views
 - query optimizer should be able to exploit them

We define the physical layout without affecting the logical one

Physical design

- **Optimizer characteristics**

- should consider statistics when defining the access plan (cost based)

- **aggregate navigation**

→ You can write a query without knowing that a view which answers that query is already available

- Physical design procedure

- selection of physical structures supporting most frequent (or most relevant) queries
- selection of structures improving performance of more than one query
- constraints
 - disk space
 - available time window for data update

Physical design

- Tuning
 - a posteriori change of physical access structures
 - workload monitoring tools are needed
 - frequently required for OLAP applications
- Parallelism
 - data fragmentation
 - query parallelization
 - inter-query
 - intra-query
 - join and group by lend themselves well to parallel execution

Index selection

- Indexing dimensions
 - attributes frequently involved in selection predicates
 - if domain cardinality is high, then B-tree index
 - if domain cardinality is low, then bitmap index
- Indices for join
 - indexing only foreign keys in the fact table is *rarely* appropriate
 - bitmapped join index is suggested (if available)
- Indices for group by
 - use materialized views

ETL Process

Elena Baralis
Politecnico di Torino

Extraction, Transformation and Loading (ETL)

- Prepares data to be loaded into the data warehouse
 - data extraction from (OLTP and external) sources
 - data cleaning
 - data transformation
 - data loading
- Eased by exploiting the staging area
- Performed
 - when the DW is first loaded
 - during periodical DW refresh

Three level architecture

A red arrow originates from the handwritten text 'Three level architecture' and points towards the 'staging area' in the list item 'Eased by exploiting the staging area'. The 'staging area' is also circled in red.

Extraction

- Data acquisition from sources → Critical step
- Extraction methods
 - static: snapshot of operational data
 - performed during the first DW population
 - incremental: selection of updates that took place after last extraction → The most complex one
 - exploited for periodical DW refresh
 - immediate or deferred
- The selection of which data to extract is based on their quality

Extraction

- It depends on how operational data is collected
 - **historical**: all modifications are stored for a given time in the OLTP system → *For critical data*
 - bank transactions, insurance data
 - operationally simple
 - **partly historical**: only a limited number of states is stored in the OLTP system → *only for a limited time window*
 - operationally complex
 - **transient**: the OLTP system only keeps the *current* data state
 - example: stock inventory
 - operationally complex

Incremental extraction

- Application assisted
 - data modifications are captured by ad hoc application functions
 - requires changing OLTP applications (or APIs for database access)
 - increases application load
 - hardly avoidable in legacy systems

- Log based
 - log data is accessed by means of appropriate APIs
 - log data format is usually proprietary
 - efficient, no interference with application load

log files support in particular deferred incremental extraction

log files are usually written in proprietary format

The use of log files introduces overhead, but the benefits gained are numerous

Transaction take a lot of times: DBMS try to reduce read and write operations postponing some of them thanks to log files

Incremental extraction

As a customer modify the database at the application level, triggers propagate changes at DWS level

- Trigger based

- triggers capture interesting data modifications
- does not require changing OLTP applications
- increases application load

The cost for trigger execution

- Timestamp based

- modified records are marked by the (last) modification timestamp
- requires modifying the OLTP database schema (and applications)
- deferred extraction, may lose intermediate states if data is transient

Timestamp is used to store informations in a particular time

Comparison of extraction techniques

	<i>Static</i>	<i>Timestamps</i>	<i>Appilcation assisted</i>	<i>Trigger</i>	<i>Log</i>
<i>Management of transient or semi-periodic data</i>	No	Incomplete	Complete	Complete	Complete
<i>Support to file-based systems</i>	Yes	Yes	Yes	No	Rare
<i>Implementation technique</i>	Tools	Tools or internal developments	Internal developments	Tools	Tools
<i>Costs of enterprise specific development</i>	None	Medium	High	None	None
<i>Use with legacy systems</i>	Yes	Difficult	Difficult	Difficult	Yes
<i>Changes to applications</i>	None	Likely	Likely	None	None
<i>DBMS-dependent procedures</i>	Limited	Limited	Variabile	High	Limited
<i>Impact on operational system performance</i>	None	None	Medium	Medium	None
<i>Complexity of extraction procedures</i>	Low	Low	High	Medium	Low

From Devlin, Data warehouse: from architecture to implementation, Addison-Wesley, 1997

Copyright – All rights reserved

DATA WAREHOUSE: DESIGN - 67

Elena Baralis
Politecnico di Torino

Incremental extraction

4/4/2010

Cod	Product	Customer	Qty
1	Greco di tufo	Malavasi	50
2	Barolo	Maio	150
3	Barbera	Lumini	75
4	Sangiovese	Cappelli	45

6/4/2010

Cod	Product	Customer	Qty
1	Greco di tufo	Malavasi	50
2	Barolo	Maio	150
4	Sangiovese	Cappelli	145
5	Vermentino	Maltoni	25
6	Trebbiano	Maltoni	150

Incremental difference

Cod	Product	Customer	Qty	Action
3	Barbera	Lumini	75	D
4	Sangiovese	Cappelli	145	U
5	Vermentino	Maltoni	25	I
6	Trebbiano	Maltoni	150	I

From Golfarelli, Rizzi, "Data warehouse, teoria e pratica della progettazione", McGraw Hill 2006

Data cleaning

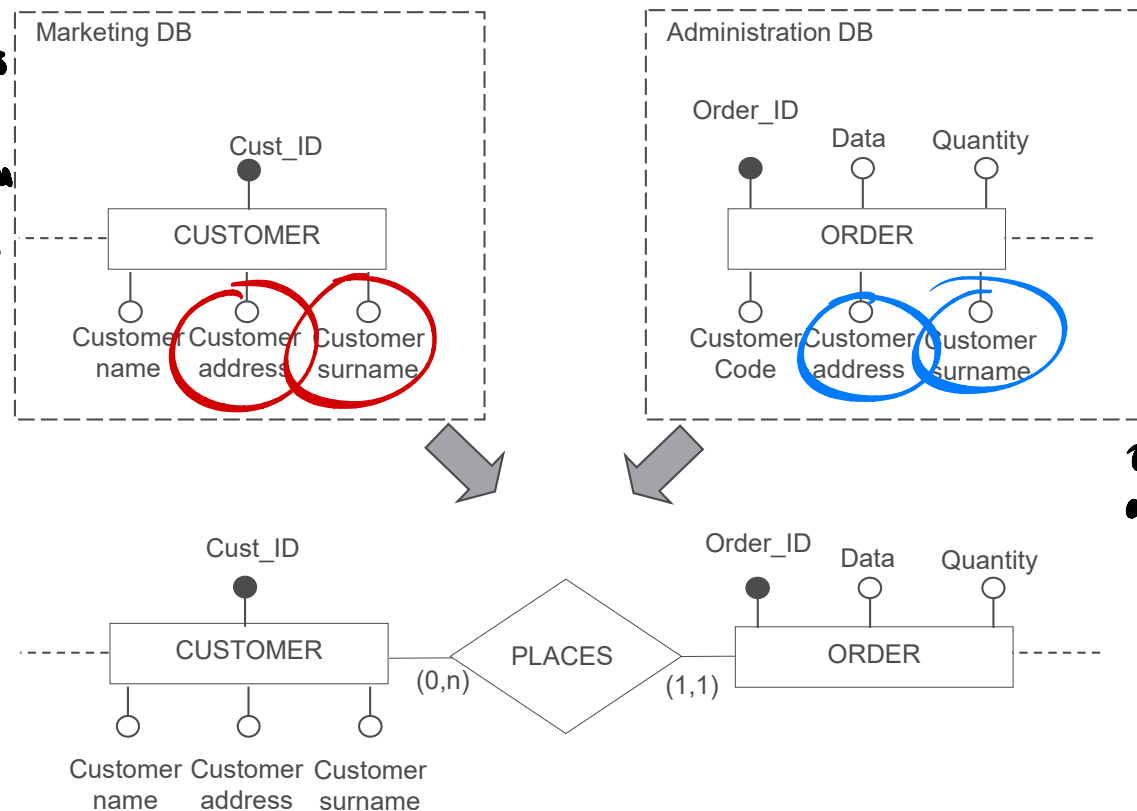
- Techniques for improving data quality (correctness and consistency)
 - duplicate data
 - missing data
 - unexpected use of a field
 - impossible or wrong data values
 - inconsistency between logically connected data
- Problems due to
 - data entry errors
 - different field formats
 - evolving business practices

Data cleaning

- Each problem is solved by an ad hoc technique
 - data dictionary
 - appropriate for data entry errors or format errors
 - can be exploited only for data domains with limited cardinality
 - approximate fusion
 - appropriate for detecting duplicates/similar data correlations
 - approximate join
 - purge/merge problem
 - outlier identification, deviations from business rules
- Prevention is the best strategy
 - reliable and rigorous OLTP data entry procedures

Approximate join

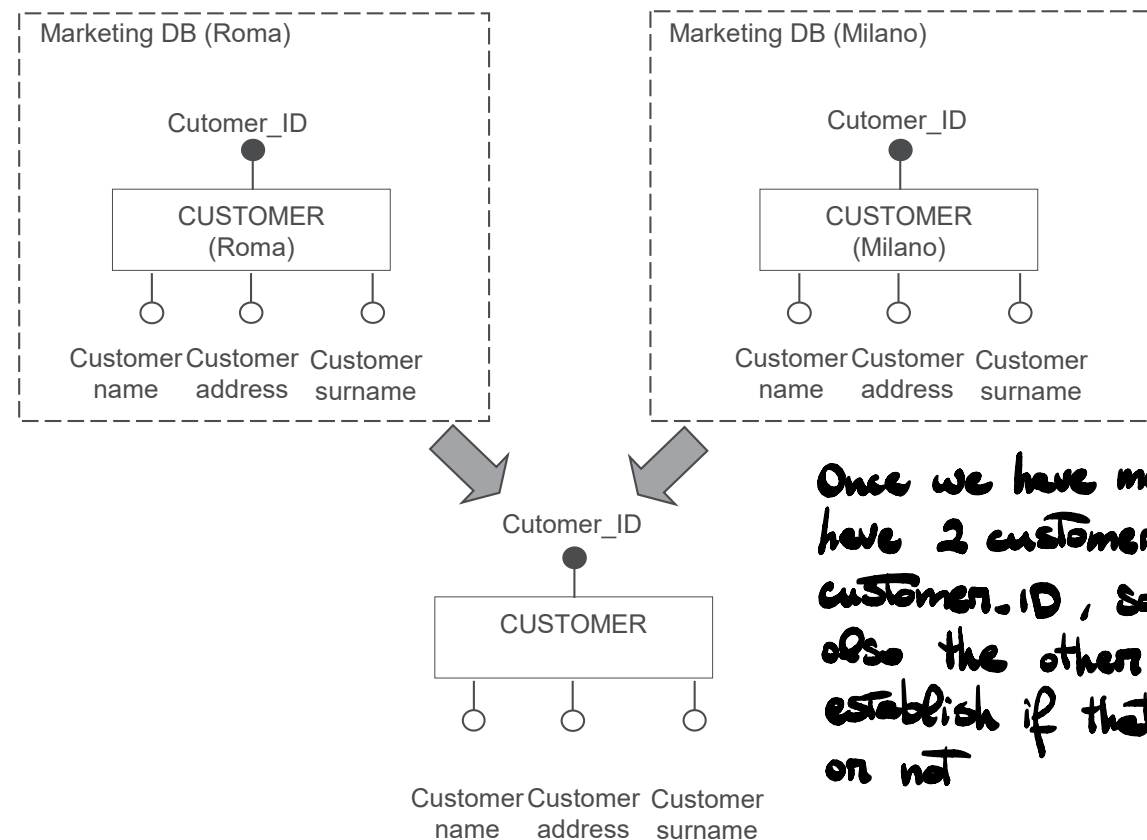
We must select the attributes in common between the 2 tables (if there are no primary keys in common, this is an approximate join)



Often, this solution is not applicable

- The join operation should be executed based on common fields, not representing the customer identifier

Purge/Merge problem



- Duplicate tuples should be identified and removed
- A criterion is needed to evaluate record similarity

From Golfarelli, Rizzi, "Data warehouse, teoria e pratica della progettazione", McGraw Hill 2006

Data cleaning and transformation example

Elena Baralis
C.so Duca degli Abruzzi 24
20129 Torino (I)

1
Normalization

name:	Elena
surname:	Baralis
address:	C.so Duca degli Abruzzi 24
ZIP:	20129
city:	Torino
country:	I

2
Standardization

name:	Elena
surname:	Baralis
address:	Corso Duca degli Abruzzi 24
ZIP:	20129
city:	Torino
country:	Italia

3
Correction

name:	Elena
surname:	Baralis
address:	Corso Duca degli Abruzzi 24
ZIP:	10129
city:	Torino
country:	Italia

Adapted from Golfarelli, Rizzi, "Data warehouse, teoria e pratica della progettazione", McGraw Hill 2006

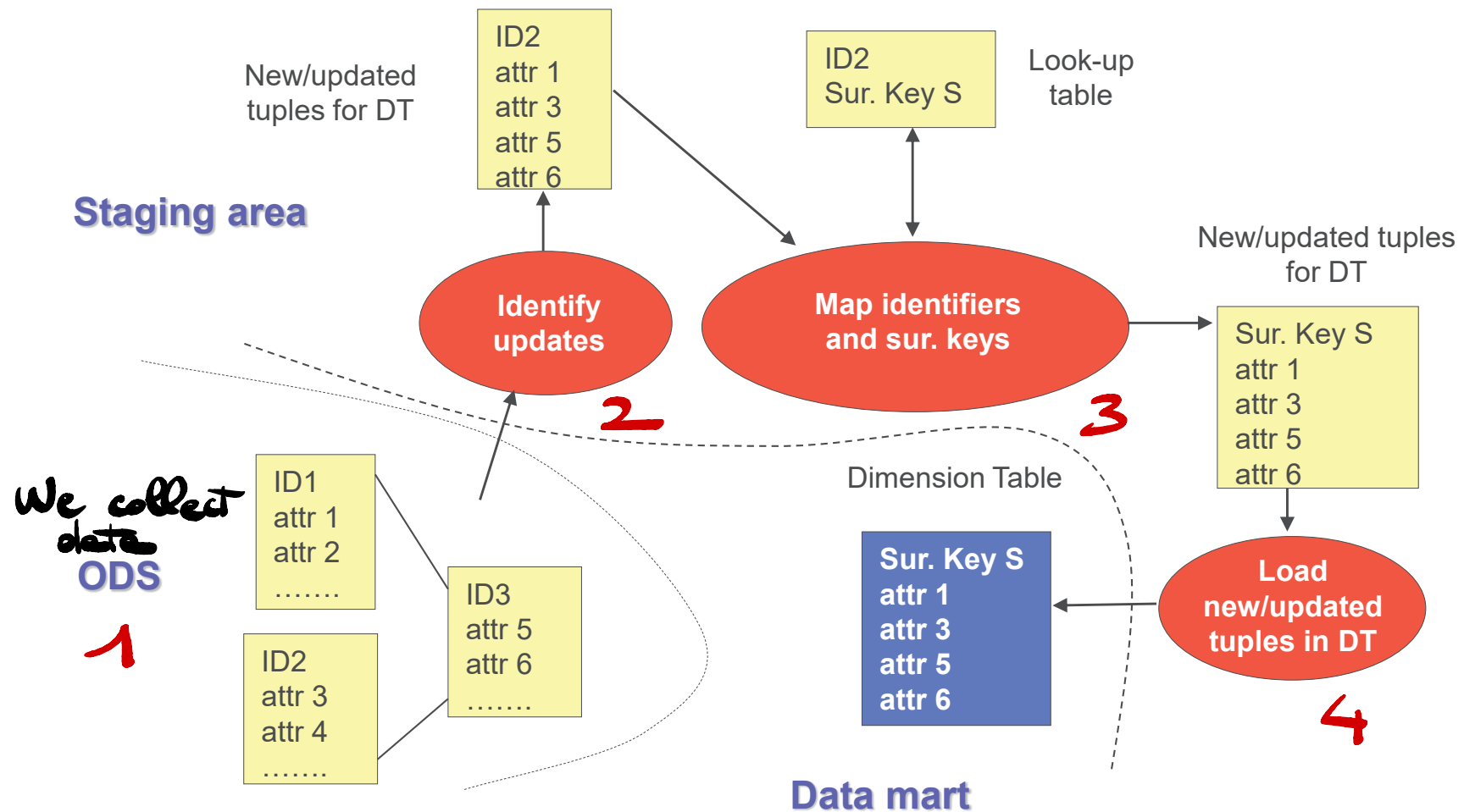
Transformation

- Data conversion from operational format to data warehouse format
 - requires data integration
- A uniform operational data representation (reconciled schema) is needed
- Two steps
 - from operational sources to reconciled data in the staging area
 - conversion and normalization
 - matching
 - (possibly) significant data selection
 - from reconciled data to the data warehouse
 - surrogate keys generation
 - aggregation computation

Data warehouse loading

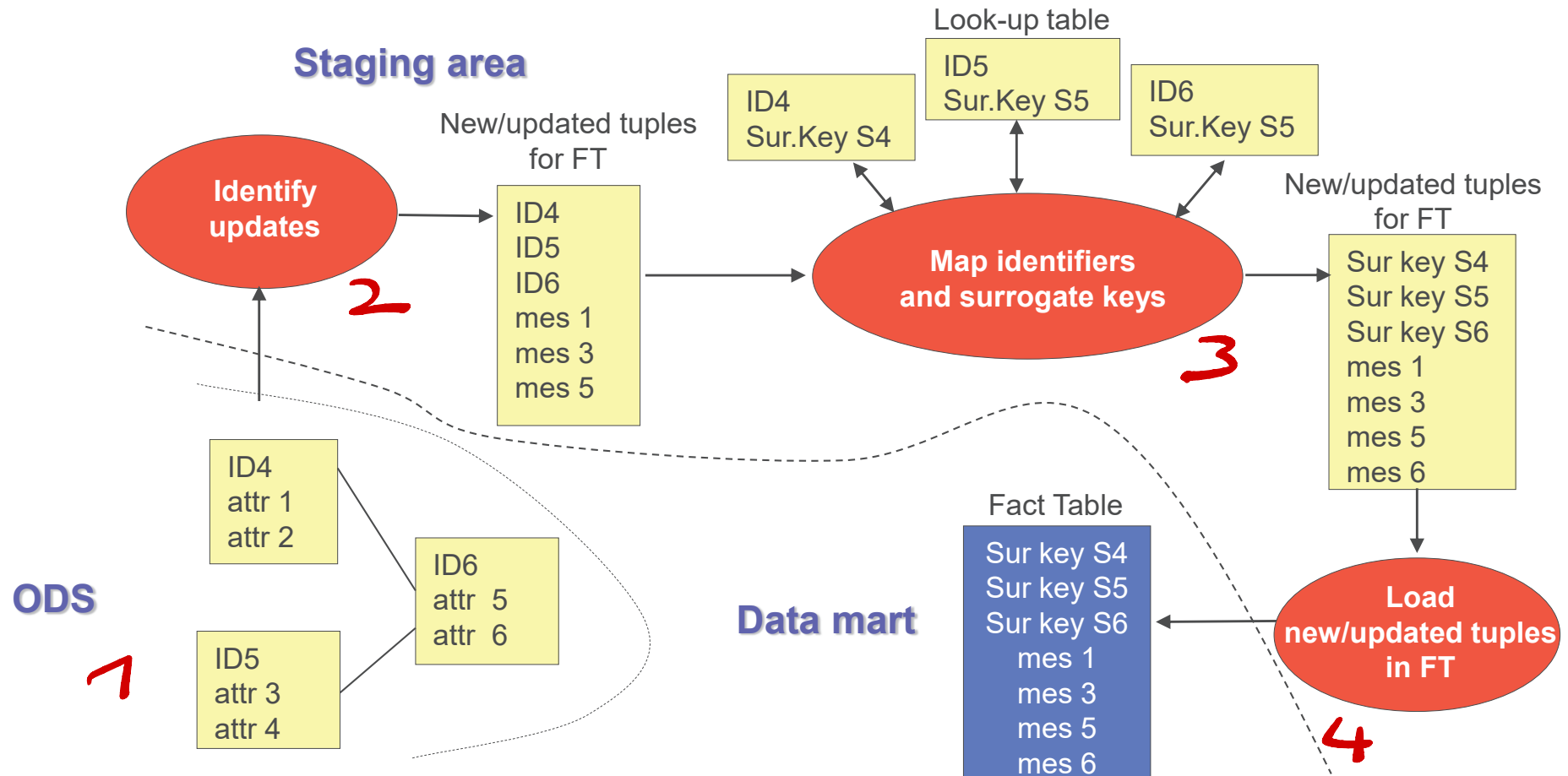
- Update propagation to the data warehouse
- Update order that preserves data integrity
 1. dimensions
 2. fact tables
 3. materialized views and indices
- Limited time window to perform updates
- Transactional properties are needed
 - reliability
 - atomicity

Dimension table loading



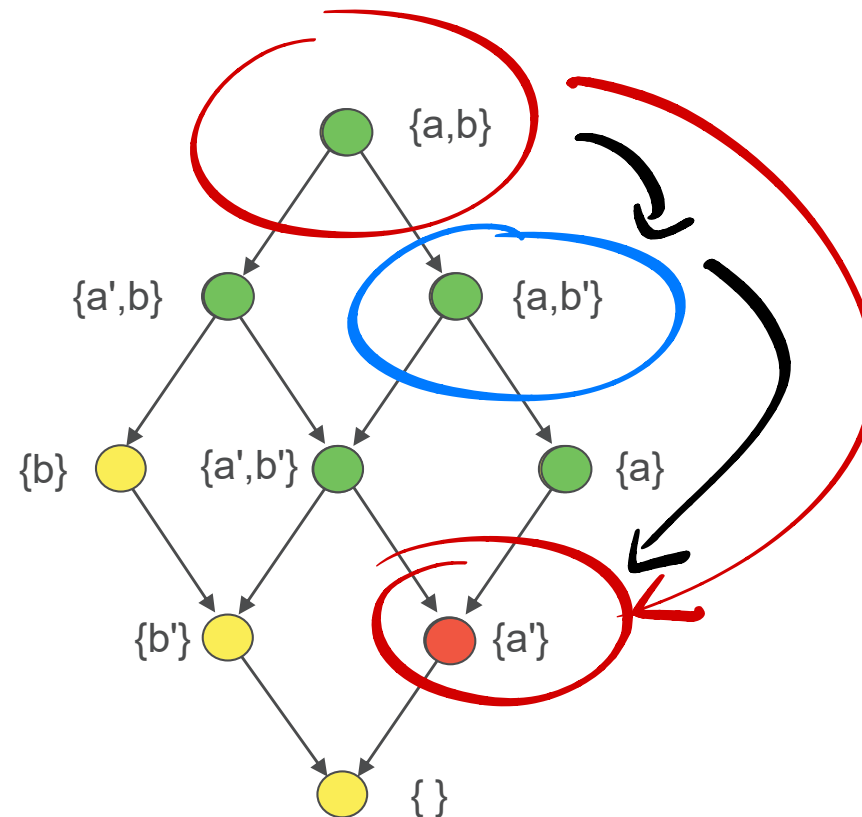
From Golfarelli, Rizzi, "Data warehouse, teoria e pratica della progettazione", McGraw Hill 2006

Fact table loading



From Golfarelli, Rizzi, "Data warehouse, teoria e pratica della progettazione", McGraw Hill 2006

Materialized view loading



Tratto da Golfarelli, Rizzi, "Data warehouse, teoria e pratica della progettazione", McGraw Hill 2006